

GROVE: Ownership Verification of Graph Neural Networks using Embeddings

Asim Waheed
University of Waterloo
asim.waheed@uwaterloo.ca

Vasisht Duddu
University of Waterloo
vasisht.duddu@uwaterloo.ca

N. Asokan
University of Waterloo
and Aalto University
asokan@acm.org

Abstract—Graph neural networks (GNNs) have emerged as a state-of-the-art approach to model and draw inferences from large scale graph-structured data in various application settings such as social networking. The primary goal of a GNN is to learn an *embedding* for each graph node in a dataset that encodes both the node features and the local graph structure around the node.

Prior work has shown that GNNs are prone to model extraction attacks. Model extraction attacks and defenses have been explored extensively in other non-graph settings. While detecting or preventing model extraction appears to be difficult, deterring them via effective *ownership verification techniques* offer a potential defense. In non-graph settings, *fingerprinting* models, or the data used to build them, have shown to be a promising approach toward ownership verification.

We present GROVE, a state-of-the-art GNN model fingerprinting scheme that, given a *target* model and a *suspect* model, can reliably determine if the suspect model was trained independently of the target model or if it is a *surrogate* of the target model obtained via model extraction. We show that GROVE can distinguish between surrogate and independent models even when the independent model uses the same training dataset and architecture as the original target model.

Using six benchmark datasets and three model architectures, we show that GROVE consistently achieves low false-positive and false-negative rates. We demonstrate that GROVE is *robust* against known fingerprint evasion techniques while remaining computationally *efficient*.

Index Terms—Graph Neural Networks, Model Extraction, Ownership Verification.

1. Introduction

Graph data is ubiquitous and used to model networks such as social networks, chemical compounds, and financial transactions. However, the non-euclidean nature of graph data makes it difficult to analyze using traditional machine-learning algorithms. Unlike Euclidean datasets, where the data points are independent, graph data follows homophily, where similar nodes share an edge. To analyze such graph data, special deep neural networks (DNNs) called Graph Neural Networks (GNNs) have been introduced [1]–[4]. These models learn an embedding for each node in the graph

that encodes the node features and local graph structure. They can perform node classification [1]–[4], link prediction [5]–[8], visualization [9], and recommendations [10]–[13].

Model builders spend significant time and resources to prepare the training data, perform hyperparameter tuning, and optimize the model to achieve state-of-the-art performance. This makes the deployed GNNs a critical intellectual property for model owners. For instance, Amazon Neptune [14] provides a framework to train and use GNNs; Facebook [15] and Twitter [16] extensively use graph-based ML models. This broad adoption of GNNs makes them vulnerable to model extraction attacks where an adversary tries to train a local *surrogate* model with similar functionality as the deployed *target* model [17]–[22].

Model extraction attacks use the input-output pairs from the target model to train the surrogate model. The goal is to achieve similar utility on the primary task for a fraction of the cost. These attacks violate the company’s intellectual property and allow further attacks such as evasion using adversarial inputs and membership inference [18]. While prior work has indicated the feasibility of model extraction attacks on various domains [18], [22]–[24], recent work introduced such attacks against GNNs as well [25]–[27].

Current approaches to address model extraction attacks rely on post-hoc *ownership verification* in which the model owner requests a trusted verifier to decide whether a *suspect model* was stolen from their target model. Ownership verification is done using either fingerprinting or watermarking. Watermarking has been shown to degrade model accuracy [28]–[30] and can be evaded [31], [32]. Hence, we identify fingerprinting as a potential scheme that can be used for model ownership verification. *Model fingerprinting* uses inherent features of the model to distinguish between surrogates and independent models. [31], [33]–[37]. On the other hand, *dataset fingerprinting* uses the training data as a fingerprint, such that any model trained on the same data as the target model is classified as stolen [32]. Such fingerprinting schemes have been proposed for non-graph DNNs, but there is currently no such work for GNNs.

In this work, we present the **first fingerprinting scheme for GNNs**. We claim the following main contributions:

- 1) identify GNN embeddings as a potential fingerprint and show that they are useful for verifying model ownership but not dataset ownership (Section 4).

- 2) present GROVE, embedding-based fingerprinting for GNN model ownership verification (Section 5).
- 3) extensively evaluate GROVE on six datasets and three architectures showing that GROVE is:
 - effective at distinguishing between surrogate and independent models with close to zero false positives or false negatives (Section 7.1),
 - robust against known fingerprint removal techniques (Section 7.2), and
 - computationally efficient (Section 7.3).

We open source our implementation: <https://github.com/ssg-research/GrOVe>

2. Background

We describe some preliminaries for GNNs and notations used in this work (Section 2.1), followed by an overview of model extraction attacks and defenses (Section 2.2).

2.1. Graph Neural Networks

Several real-world applications can be modeled as graphs that include nodes that represent different entities in the graph (e.g., authors in citation networks or users in social networks) and edges that represent the connections between nodes. Formally, a graph can be represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of nodes and $\mathcal{E}$ is a set of edges connecting these nodes. We represent a single node as $v \in \mathcal{V}$ and an edge between nodes u and v as $e_{uv} \in \mathcal{E}$. The entire graph structure, including all the nodes and corresponding edges, can be represented using a binary adjacency matrix $\mathcal{A}$ of size $|\mathcal{V}| \times |\mathcal{V}|$: $\mathcal{A}_{uv} = 1, \forall (e_{uv}) \in \mathcal{E}$.

ML on Graphs. Due to the large scale of these graph datasets, ML approaches for processing them have gained significant attention. Specifically, GNNs have shown tremendous performance in processing graph data for node and edge classification, clustering, and other tasks. Following prior work [25]–[27], we consider node classification tasks in this work.

Each node has a feature vector $x \in \mathcal{X}$ and corresponding classification label $y \in \mathcal{Y}$ where $\mathcal{X}$ and $\mathcal{Y}$ are the set of features and labels across all nodes respectively. We can denote the graph dataset as $\mathcal{D} = (\mathcal{A}, \mathcal{X}, \mathcal{Y})$ which is a tuple of the adjacency matrix, the set of node features, and the set of labels respectively.

GNNs are trained to take the graph’s adjacency matrix and the features as input and map it to the corresponding classification labels. Once trained, GNNs output a node embedding $h \in \mathcal{H}$. Embeddings are low-dimensional representations of each node and the graph structure and can be used for downstream tasks such as classification, recommendations, etc. We note that since $\mathcal{H}$ is dependent on the graph structure and the node features, two GNNs trained on different datasets should differ in their output of $\mathcal{H}$ [2], [4]. Formally, the mapping for a GNN is written as: $\mathcal{F} : \mathcal{A} \times \mathcal{X} \rightarrow \mathcal{H}$.

There are two training paradigms for GNNs:

- **Transductive** where the model trains on mapping some graph nodes to classification labels but uses the remaining graph nodes for prediction during testing. Here, the underlying graph structure $\mathcal{A}$, passed as input to the model, remains the same. This is useful for labeling a partially-labeled graph.
- **Inductive** where the model is trained on a training graph dataset but evaluated on an unseen and disjoint testing dataset.

Following prior work [27], we consider the more prevalent and practical setting of inductive training, which is applicable for ML as a service on graph data.

GNN Computation. Training and evaluating a GNN involves aggregating information from neighboring nodes to compute the embeddings for a specific node. Formally, each layer of a GNN performs the following operation:

$$h_v^l = \text{AGG}(h_v^{l-1}, \text{MSG}(h_v^{l-1}, h_u^{l-1} : u \in \mathcal{N}(v))) \quad (1)$$

$\mathcal{N}(v)$ denotes the nodes that share an edge with v . h_v^l denotes the embedding of node v at layer l . h_v^0 is initialized as the feature vector x for node v . $\text{MSG}(\cdot)$ gathers information from the neighbouring nodes of v , and $\text{AGG}(\cdot)$ aggregates this information with h_v^{l-1} to produce h_v^l . The primary difference between different GNN architectures is the implementation of these two functions.

GraphSAGE [2] uses the mean aggregation operation:

$$h_v^l = \text{CONCAT}(h_v^{l-1}, \text{MEAN}(h_u^{l-1} : u \in \mathcal{N}(v))) \quad (2)$$

where CONCAT is the concatenation operation, and MEAN is the mean operation.

Graph Attention Networks (GAT) [4] includes masked self-attention layers in the GNN, which allows the model to assign a different weight to each neighbor of a node. This captures the variation in the contribution of different neighboring nodes. The aggregation function is:

$$h_v^l = \text{CONCAT}_{k=1}^K \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv}^k \mathbf{W}^k h_u^{l-1} \right) \quad (3)$$

where CONCAT is the concatenation operation, K is the total number of projection heads in the attention mechanism, α_{uv}^k is the attention coefficient in the k^{th} projection head, $\mathbf{W}^k$ is the linear transformation weight matrix, and $\sigma(\cdot)$ is the activation function.

Graph Isomorphism Network (GIN) [3] extends GraphSAGE and uses the aggregation function:

$$h_v^l = \text{MLP}^l((1 + \epsilon^l) \cdot h_v^{l-1} + \sum_{u \in \mathcal{N}(v)} h_u^{l-1}) \quad (4)$$

where MLP is a multi-layer perceptron and ϵ is a learnable parameter to adjust the weight of node v . This treats $h_u^{l-1} : u \in \mathcal{N}(v)$ as a *multiset*, i.e., a set with possible repeating elements.

2.2. Model Extraction Attacks and Defences

Model extraction attacks consider an adversary (Adv) who trains a local surrogate model ($\mathcal{F}_s$) to mimic the functionality of a target model ($\mathcal{F}_t$) [17]. These attacks have been extensively studied in many domains including images [18]–[20], text [24], [38], and graphs [25]–[27] and across different types of models including generative models [23], [39], and large language models [40]. This attack has been identified as a realistic threat that violates the confidentiality of the company’s proprietary model and thereby their intellectual property [22].

We denote the training dataset of $\mathcal{F}_t$ (respectively $\mathcal{F}_s$) as $\mathcal{D}_t$ ($\mathcal{D}_s$). Additionally, we refer to models trained independently on a dataset $\mathcal{D}_i$ (in the absence of model extraction attack) are called independent models ($\mathcal{F}_i$).

Non-Graph Model Extraction Attacks. Adv , given query access to $\mathcal{F}_t$, sends queries and obtains corresponding predictions. Adv then uses these input-prediction pairs to train $\mathcal{F}_s$. Most attacks train $\mathcal{F}_s$ using specially crafted adversarial examples as inputs to $\mathcal{F}_t$ [17]–[20]. This helps to ensure that the decision boundary of $\mathcal{F}_s$ is similar to $\mathcal{F}_t$. After a successful attack, Adv can use $\mathcal{F}_s$ to generate effective transferrable adversarial examples [18] or perform membership inference attacks [41].

Non-Graph Model Extraction Defenses. Preventing model extraction attacks without affecting model performance is difficult [21], [22], [38]. However, ownership verification as a post-hoc approach helps identify whether a suspect model ($\mathcal{F}_?$) is stolen via model extraction. Normally, this involves a third-party verifier (Ver) that verifies ownership. There are currently two main schemes for ownership verification:

- *watermarking* [42]–[51] where some secret information is embedded into the model during training which is extracted later during verification.
- *fingerprinting* [31]–[34], [36], [37] where inherent features are extracted from a model, without affecting the training process.

Prior work has shown watermarking is brittle and can be easily evaded [52]. Hence, fingerprinting is currently the most promising technique for ownership verification. In this work, we focus on fingerprinting.

Prior work has explored two fingerprinting schemes based on whether the fingerprint is for *dataset ownership* or *model ownership*. The subtle difference between them is how a *different* model trained from scratch on *the same dataset* as $\mathcal{F}_t$ is treated. Dataset-ownership-based fingerprinting classifies such a model as a surrogate. However, model-ownership-based fingerprinting classifies such a model as independent. Here, only models derived (e.g., transfer-learning, model extraction) from $\mathcal{F}_t$ are classified as a surrogate. We describe the main prior works for fingerprinting below.

For dataset ownership-based fingerprinting, **Maini et al.** [32] propose *dataset inference* on the following intuition: the distance of the training data points from the model’s decision boundary is increased during training. Hence, the

distance of a data point from the decision boundary can help infer its membership in $\mathcal{D}_t$. Ver queries $\mathcal{F}_?$ with data points from $\mathcal{D}_t$ and unseen public data to compute their distances from the decision boundary. $\mathcal{F}_?$ is classified as a surrogate if the distances corresponding to $\mathcal{D}_t$ are large, and the distances corresponding to unseen public data are small.

Most prior work on model ownership-based fingerprinting identifies adversarial examples that can transfer from $\mathcal{F}_t$ to $\mathcal{F}_s$ but not to $\mathcal{F}_i$ [31], [33]–[35]. They vary in how to identify such adversarial examples. For instance, data points close to the decision boundary can be used to differentiate between $\mathcal{F}_s$ and $\mathcal{F}_i$ models [34], [35]. Alternatively, untargeted adversarial examples can be used as well [33]. However, these works consider $\mathcal{F}_s$ derived using transfer-learning and fine-tuning but do not consider model extraction attacks [31].

Two prior model ownership-based fingerprinting works are evaluated explicitly with respect to model extraction attacks. We describe them below.

Lukas et al. [31] use an ensemble of models to generate *conferrable adversarial examples*, i.e., adversarial examples that transfer from $\mathcal{F}_t$ to $\mathcal{F}_s$, but are not misclassified by $\mathcal{F}_i$. During verification, Ver computes the error between the predictions of $\mathcal{F}_t$ and $\mathcal{F}_?$ on the conferrable examples. If the error rate exceeds a certain threshold, it is classified as independent, and surrogate otherwise.

Peng et al. [36] use *universal adversarial perturbations* (UAPs), small imperceptible perturbations which, when added to any image, result in misclassification. They compute a fingerprint by adding UAP to some data points and compute the change in output before and after UAP. They train an encoder to ensure that the fingerprints of $\mathcal{F}_t$ and $\mathcal{F}_s$ are similar but distinct from $\mathcal{F}_i$.

We explain in Section 3.4 why these techniques ([31], [32], [36]) are not easily applicable to GNNs.

Model Extraction Attacks on GNNs. There is limited literature on model extraction attacks in GNNs. One approach is to use adversarial examples to perform model extraction attacks, similar to the work in non-graph settings [25]. However, the input perturbation for adversarial examples in graph setting is too high to be stealthy. Wu et al. [26] presented seven attacks with different Adv background knowledge. However, these works are limited to transductive training, impractical in ML as a service setting [27].

We focus on the more practical case of inductive learning where the training and testing graph datasets are disjoint. Shen et al. [27] presented the first work on model extraction against inductive GNNs. Here, Adv has access to a query dataset $\mathcal{D}_s = (\mathcal{A}_s, \mathcal{X}_s, \mathcal{Y}_s)$ and the query response (a set of embeddings ($\mathcal{H}_t$) corresponding to the node features ($\mathcal{X}_s$) it receives from $\mathcal{F}_t$). Using $\mathcal{H}_t$ and the ground-truth labels ($\mathcal{Y}_s$), Adv trains $\mathcal{F}_s$ to mimic the behavior of $\mathcal{F}_t$. They propose two attacks depending on Adv ’s background knowledge: in *Type I* attack, Adv has access to $\mathcal{X}_s$, $\mathcal{Y}_s$, and the adjacency matrix ($\mathcal{A}_s$) for $\mathcal{D}_s$, in *Type II* attack, Adv only has access to $\mathcal{X}_s$, $\mathcal{Y}_s$, and uses an edge-estimation algorithm to compute $\mathcal{A}_s$.

In their attack, the architecture for $\mathcal{F}_s$ consists of two components:

- a GNN that takes $\mathcal{X}_s$ and $\mathcal{A}_s$ as input and outputs $\mathcal{H}_s$. While training, this module minimizes the mean squared error (MSE) loss between $\mathcal{H}_s$ and $\mathcal{H}_t$:

$$\mathcal{H}_s = \text{Gnn}(\mathcal{X}_s, \mathcal{A}_s) \quad (5)$$

$$\mathcal{L}_R = \frac{1}{n_{D_s}} \|\mathcal{H}_s - \mathcal{H}_t\|_{2,1} \quad (6)$$

where n_{D_s} is the number of nodes in $\mathcal{D}_s$.

- an MLP classifier ($\mathcal{C}$) which takes $\mathcal{H}_s$ as input and outputs a class. This is trained to minimize the prediction loss between $\mathcal{V}_s$ and the predicted labels.

In each epoch, Gnn is first optimized using $\mathcal{L}_R$; then, the parameters are fixed while parameters corresponding to $\mathcal{C}$ are optimized. Both modules are combined to form $\mathcal{F}_s$.

The same training strategy is used to train $\mathcal{F}_s$ using Type I and Type II attacks. However, for Type II attacks, Adv first estimates $\mathcal{A}_s$. A graph structure is initialized by creating a k -nearest neighbors graph from $\mathcal{X}_s$. A search algorithm looks for hidden graph structure that augments the initial k -nearest neighbors graph [26]. The graph structure is obtained by minimizing a joint loss function that combines prediction loss for node classification and a graph regularization loss that controls the graph's smoothness, connectivity, and sparsity. Additional details about the model extraction attack can be found in [27].

Model Extraction Defenses For GNNs. To the best of our knowledge, there are two prior works on watermarking in the context of GNNs [53], [54]. Zhao et al. [54] embed randomly generated subgraphs with random feature vectors as a key. These can be used later to extract the watermark. However, they only focus on node classification tasks. Xu et al. [53] extend the prior work by including graph classification tasks. There are no known fingerprinting schemes for GNNs in the current literature.

TABLE 1. SUMMARY OF NOTATIONS USED IN THIS WORK.

Notation	Description
$\mathcal{D} = (\mathcal{A}, \mathcal{X}, \mathcal{Y})$	A graph dataset
$\mathcal{A}$	Adjacency matrix
$\mathcal{X}$	Set of node features x
$\mathcal{Y}$	Set of labels y
$\mathcal{H}$	Set of node embeddings h
e_{uv}	An edge between nodes u and v
$\mathcal{N}(v)$	Neighbouring nodes of v
$\mathcal{F}_t$	Target model
$\mathcal{F}_s$	Surrogate model
$\mathcal{F}_i$	Independent Model
$\mathcal{F}_?$	Suspect model to be verified
$\mathcal{D}_t / \mathcal{D}_s / \mathcal{D}_i$	Target / Surrogate / Independent dataset
Adv	Adversary
Ver	Verifier
Accuser	Owner of $\mathcal{F}_t$
Responder	Owner of $\mathcal{F}_?$
$\text{Adv.}\mathcal{R}$	Malicious Responder
$\text{Adv.}\mathcal{A}$	Malicious Accuser
$c_t / c_?$	Commitment of $\mathcal{F}_t / \mathcal{F}_?$
$t_t / t_?$	Timestamp of $\mathcal{F}_t / \mathcal{F}_?$

3. Problem Statement

Our goal is to design an effective fingerprinting scheme that allows Ver to identify whether $\mathcal{F}_?$ is a surrogate of $\mathcal{F}_t$ or an independent model. We consider the state-of-the-art model extraction attacks in an inductive setting [27] to evaluate our scheme. To this end, we outline a system model that defines the interactions between model owners and Ver (Section 3.1), an adversary model describing Adv 's capabilities and goals (Section 3.2), requirements to design an ideal fingerprinting scheme (Section 3.3) and limitations of prior defenses against GNN model extraction (Section 3.4). Table 1 summarizes the notations used in this work.

3.1. System Model

We consider a setting where a proprietary GNN model ($\mathcal{F}_t$) has been developed and deployed as a service. However, $\mathcal{F}_t$ is susceptible to model extraction. An ownership verification system, intended to thwart model extraction, consists of three actors: an Accuser (the owner of $\mathcal{F}_t$), a trusted third party Ver , and a Responder (the owner of a suspect model $\mathcal{F}_?$) who is accused of stealing $\mathcal{F}_t$ by Accuser . The role of the Ver is to verify whether $\mathcal{F}_?$ was obtained through a model extraction attack on $\mathcal{F}_t$. We refer to a malicious Responder as $\text{Adv.}\mathcal{R}$ and a malicious Accuser as $\text{Adv.}\mathcal{A}$.

System Design Goals. An ideal system must be robust against both $\text{Adv.}\mathcal{R}$ and $\text{Adv.}\mathcal{A}$.

Case 1 $\text{Adv.}\mathcal{R}$ wants its model $\mathcal{F}_?$, extracted from Accuser 's $\mathcal{F}_t$, to evade detection.

Case 2 $\text{Adv.}\mathcal{A}$ wants to maliciously claim that Responder 's $\mathcal{F}_?$ is extracted from $\mathcal{F}_t$.

To address both scenarios, similar to prior work [37], [51], we assume that all model owners are required to securely timestamp their models in a registration step before deployment. This will address $\text{Adv.}\mathcal{A}$ since Accuser cannot successfully make an ownership claim against $\mathcal{F}_?$ unless $\mathcal{F}_?$ was registered prior to $\mathcal{F}_?$. In Section 9, we explore possible incentives for model owners to register their models. In the rest of this paper, we will focus on robustness against a malicious $\text{Adv.}\mathcal{R}$.

Model Registration. Model owners are required to:

- 1) generate a cryptographic commitment c of their model such that any subsequent modification to the model can be detected. One way to compute such a commitment is using a cryptographic hash function.
- 2) obtain a secure timestamp t on c that Ver can later verify, e.g., by adding c to a blockchain, or utilizing a publicly verifiable timestamping service¹, or receiving a signature from Ver binding c to the current time.

We use subscripts to associate timestamps and commitments to the respective models (e.g., t_t is the secure timestamp on the commitment c_t of $\mathcal{F}_t$)

Dispute Initiation. Accuser initiates a dispute by submitting c_t and t_t to Ver , and identifying a suspect Responder .

1. E.g., <https://www.surety.com/digital-copyright-protection>

$\mathcal{V}er$ then asks $\mathcal{R}esponder$ to submit c_t and t_t to begin the verification process.

Verification Process. $\mathcal{V}er$ does the following:

- 1) verifies that t_t , c_t , and $\mathcal{F}_t$ are consistent; and t_t , c_t , and $\mathcal{F}_t$ are consistent.
- 2) confirms that $t_t < t_t$. If this condition is not met, the claim is rejected.
- 3) checks that $\mathcal{F}_t$ and $\mathcal{F}_t$ are well-formed (see below).
- 4) samples a verification dataset $\mathcal{D}_v$ from the same distribution as $\mathcal{D}_t$.
- 5) queries $\mathcal{F}_t$ and $\mathcal{F}_t$ with $\mathcal{D}_v$ and passes the outputs to a verification algorithm which decides whether $\mathcal{F}_t$ is a surrogate of $\mathcal{F}_t$ or trained independently.

Step 3 requires $\mathcal{V}er$ to check that a model does not have any non-standard layers. In Section 7.2, we explain why this check is necessary.

The simplest way for $\mathcal{V}er$ to conduct the checks in the first three steps is for the model owners to send their models ($\mathcal{F}_t$ and $\mathcal{F}_t$) to $\mathcal{V}er$. This may not be feasible for confidentiality or privacy reasons. Thus, the checks can be done either via cryptographic techniques like oblivious inference [55], [56] in conjunction with zero-knowledge proofs [57], [58] or by using hardware-based trusted execution environments [59], [60]. The specific implementation of such protocols is out of the scope of this work. For ease of exposition, we limit our discussion to the case where the model owners in a dispute are willing to share their models with $\mathcal{V}er$. Note that it is still necessary to check that the models sent to $\mathcal{V}er$ are indeed the models that were deployed. $\mathcal{V}er$ can do this via a fidelity check [20].

3.2. Adversary Model

$\mathcal{A}dv.\mathcal{R}$'s **goal** is to train $\mathcal{F}_s$ such that its utility is comparable to $\mathcal{F}_t$. Additionally, $\mathcal{A}dv.\mathcal{R}$ wants high *fidelity* for $\mathcal{F}_s$, i.e., that its inferences match $\mathcal{F}_t$'s. This is useful for mounting subsequent evasion or membership inference attacks against $\mathcal{F}_t$ [18]. Shen et al.'s [27] attack satisfies both these requirements. $\mathcal{A}dv.\mathcal{R}$ may also take additional steps to evade detection.

Attack Setting. Following [27], we consider the *black-box* setting where $\mathcal{A}dv.\mathcal{R}$ has no knowledge of $\mathcal{F}_t$'s hyperparameters or architecture and can only observe $\mathcal{F}_t$'s outputs for given inputs. As in prior work [27], we assume that the output contains node embeddings ($\mathcal{H}$), useful for downstream tasks such as classification, recommendation engines, visualizations, etc. We assume that $\mathcal{A}dv.\mathcal{R}$ has access to a dataset $\mathcal{D}_s$ from the same distribution as $\mathcal{F}_t$'s training data $\mathcal{D}_t$. However, $\mathcal{D}_s$ and $\mathcal{D}_t$ are disjoint. We revisit the details of dataset splits in Section 6.

3.3. Requirements for Ownership Verification

We list desiderata for ideal GNN ownership verification schemes that $\mathcal{V}er$ can use to decide if $\mathcal{F}_t$ is stolen from $\mathcal{F}_t$:

R1 Minimize Utility Loss of $\mathcal{F}_t$.

R2 Effective in differentiating between $\mathcal{F}_s$ and $\mathcal{F}_i$ with low false positive/negative rates.

R3 Robust in remaining effective against $\mathcal{A}dv.\mathcal{R}$.

R4 Efficient by imposing a low computational overhead.

3.4. Limitations of Prior Work

We now discuss how different prior works (Section 2.2), applicable to non-graph and graph datasets, do not satisfy the above requirements.

Non-Graph Datasets. Focusing only on approaches tested against model extraction attacks, we discuss how the three prior fingerprinting approaches for the image domain are not directly applicable to GNNs. We describe the limitations of prior non-graph fingerprinting schemes below.

Maini et al. [32] compute the distance of a data point to the decision boundary by adding noise to the data points, which is not clear for inter-connected graph nodes [25]. Moreover, prior works have indicated that dataset inference incurs false positives [48], [61]. Lukas et al. [31] and Peng et al. [36] rely on adversarial examples as fingerprints. However, unlike images, generating adversarial examples is not trivial for graphs [62], [63].

In summary, adapting non-graph fingerprinting approaches to graph datasets is not trivial, as data records in the image domain are independent. In contrast, nodes in graphs are related and satisfy homophily.

Graph Datasets. Watermarking schemes have been proposed previously for GNNs [53], [54]. However, watermarks in non-graph datasets can be easily removed by model extraction attacks and are hence not robust **R3** [52]. Their effectiveness **R2** against state-of-the-art model extraction attacks is not clear. Finally, watermarks require modifying how the model is trained, violating the non-invasive requirement **R1**. Hence, in this work, we focus on using fingerprints for ownership resolution in GNNs instead of watermarking, which satisfies all the desirable requirements.

4. Can GNN embeddings serve as fingerprints?

We now motivate our use of GNN embeddings as a potential fingerprinting scheme for GNNs against model extraction attacks. We then evaluate whether embeddings are helpful for model fingerprinting or dataset fingerprinting.

Embeddings as Fingerprint for GNNs. Recall from Section 2.1 that two independently trained GNNs should differ in their output of $\mathcal{H}$. Furthermore, the state-of-the-art model extraction attack against GNNs [27] focuses on optimizing *fidelity*, i.e., ensuring alignment in predictions between $\mathcal{F}_s$ and $\mathcal{F}_t$. Hence, $\mathcal{F}_s$ is likely to generate embeddings more similar to $\mathcal{F}_t$ on the same input graph than $\mathcal{F}_i$. This intuition forms the basis of our fingerprinting scheme, GROVE, which uses node embeddings as a fingerprint to distinguish between $\mathcal{F}_s$ and $\mathcal{F}_i$ for ownership verification. Note that $\mathcal{H}$ is inherent to GNN model computation. Hence, they do not affect the utility of the GNN, satisfying requirement **R1**.

Model Ownership vs. Dataset Ownership. Having identified graph embeddings as a potential fingerprint, we want

to verify whether they are helpful for model or dataset ownership. Recall from Section 2.2 that the difference in fingerprinting for model ownership and dataset ownership is in how models trained on the same training dataset are classified. If embeddings generated from two GNNs trained on the same dataset cannot be distinguished, then embeddings are useful as fingerprints for dataset ownership. On the other hand, if embeddings generated from $\mathcal{F}_s$, regardless of the training data, can be distinguished, they are helpful as fingerprints for model ownership. We test this using t-SNE projections of the embeddings to visualize them for different model architectures and datasets [36], [64]. We refer to embeddings generated from $\mathcal{F}_t$, $\mathcal{F}_s$, and $\mathcal{F}_i$ as $\mathcal{H}_t$, $\mathcal{H}_s$, and $\mathcal{H}_i$, respectively.

We describe our experimental setup to infer whether embeddings can be used for data or model ownership verification.

Dataset Ownership. We train two models using either different or the same architecture and training dataset. When the training datasets are different, we split the dataset into three sets: $\mathcal{D}_1$ and $\mathcal{D}_2$, and a verification dataset $\mathcal{D}_v$. We use three different architectures for each model, leading to nine pair-wise combinations. We use six datasets, resulting in 54 model pairs with the same training datasets and 54 model pairs with differing training datasets.

We pass $\mathcal{D}_v$ to both models to generate embeddings and visualize their t-SNE projections. The graphs for COAUTHOR are shown in Figure 1, and the rest of the graphs are in Appendix A. We found that in every case, the t-SNE projections of the embeddings generated from $\mathcal{D}_v$ are distinguishable, even when the two models use the same dataset and architecture. This shows us that embeddings may not be useful as dataset fingerprints.

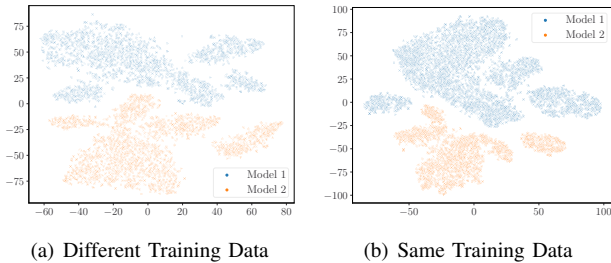
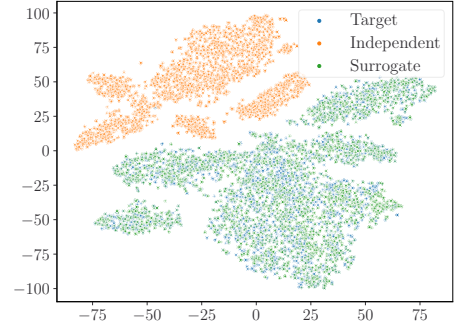


Figure 1. t-SNE projections of the embeddings from two models trained on COAUTHOR using both the same and different architectures. The embeddings are distinguishable even when the architecture is the same. The graphs for the other datasets can be found in Appendix A

Model Ownership. To check for model ownership, we use three models: $\mathcal{F}_t$, $\mathcal{F}_s$, $\mathcal{F}_i$, which may or may not share the same architecture and training data as $\mathcal{F}_t$. We set up a similar experiment to the one before. We split the dataset into two training datasets: $\mathcal{D}_1$ and $\mathcal{D}_2$. $\mathcal{D}_1$ is used to train both $\mathcal{F}_t$ and $\mathcal{F}_i$ since this is the worst-case scenario for triggering false accusations using the fingerprinting scheme. $\mathcal{F}_s$ is derived from $\mathcal{F}_t$ using $\mathcal{D}_2$ with Shen et al.’s [27] state-of-the-art GNN model extraction attack described in Section 2.2. Similar to the previous experiment, we build

multiple combinations of the three models. We use three architectures for $\mathcal{F}_t$ and $\mathcal{F}_i$, and two for $\mathcal{F}_s$, resulting in 108 model combinations across six datasets.

We plot the embeddings of $\mathcal{D}_v$ from each of the three models. We show two of the graphs from COAUTHOR and DBLP in Figure 2 and the rest in Appendix 7. We found that regardless of the architecture or the training data, the t-SNE projections of $\mathcal{H}_t$ and $\mathcal{H}_s$ are similar but distinct from $\mathcal{H}_i$. This motivates our choice to use embeddings as a fingerprint for model ownership.



(a) Models trained on COAUTHOR dataset



(b) Models trained on DBLP dataset

Figure 2. t-SNE projections of the embeddings from $\mathcal{F}_s$ and $\mathcal{F}_t$ overlap, while those from $\mathcal{F}_i$ are distinct. The models in this plot are all trained with the GAT architecture. The plots for the rest of the datasets are in Appendix B

5. GROVE: Fingerprinting for GNN Ownership Verification

We now describe the design of GROVE, which uses embeddings as fingerprints.

Recall that $\mathcal{V}_{er}$ aims to identify whether $\mathcal{F}_?$ was obtained via a model extraction attack on $\mathcal{F}_t$ (Section 3.1). As shown in Section 4, we know that the embeddings generated by $\mathcal{F}_t$ and $\mathcal{F}_s$ are similar. Our verification scheme relies on this observation and identifies whether the distances between $\mathcal{H}_?$ (generated by $\mathcal{F}_?$) and $\mathcal{H}_t$ are *close enough* to suggest a model extraction attack.

The simplest way to identify this is to calculate a distance metric between $\mathcal{H}_?$ and $\mathcal{H}_t$. If the aggregated distances

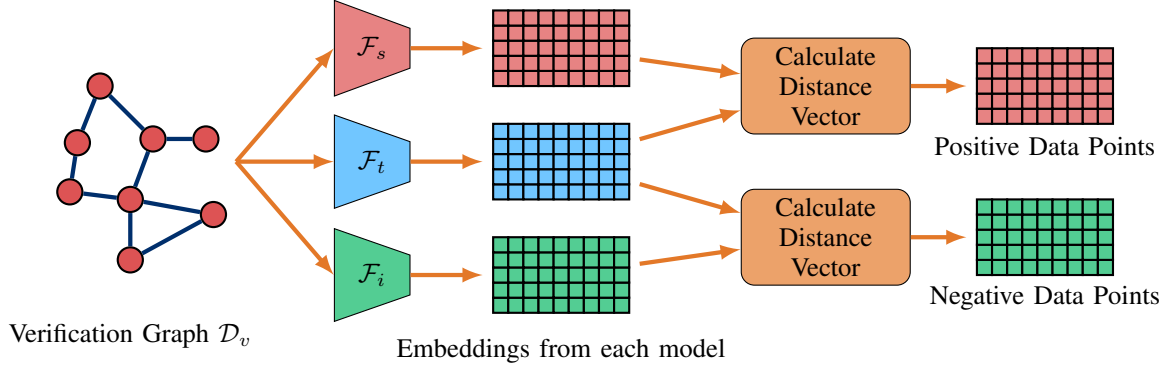


Figure 3. To generate training data for $\mathcal{C}_{sim}$, an input “verification” graph is passed to $\mathcal{F}_t$, $\mathcal{F}_s$ and $\mathcal{F}_i$. We then compute the distance vectors between the embeddings obtained from each of the models, namely, $(\mathcal{F}_t, \mathcal{F}_s)$ and $(\mathcal{F}_t, \mathcal{F}_i)$ which act as positive and negative data points respectively.

are smaller than a tuned threshold, we can classify $\mathcal{F}_?$ and $\mathcal{F}_s$, and $\mathcal{F}_i$ otherwise. However, our experiments found that the distances between $(\mathcal{F}_t, \mathcal{F}_i)$ pairs and $(\mathcal{F}_t, \mathcal{F}_s)$ pairs overlapped, leading to high error rates. This phenomenon is visualized in the distance plots in Appendix C.

Thus, similar to the intuition used for Siamese Networks [65], we use an ML classifier to classify a pair of embeddings as *similar* or *not similar*. An ML-based approach compresses the distance between similar data points and stretches the distance between dissimilar data points. The ML classifier is a multi-layer perceptron, denoted as $\mathcal{C}_{sim}$. Our verification scheme is divided into two phases: Phase 1 involves training $\mathcal{C}_{sim}$; Phase 2 includes querying $\mathcal{C}_{sim}$ with pairs of embeddings generated by $\mathcal{F}_?$ and $\mathcal{F}_t$ on $\mathcal{D}_v$ to classify $\mathcal{F}_?$ as either a surrogate or independent model.

$\mathcal{C}_{sim}$ Training. Figure 3 shows the process of generating training data for $\mathcal{C}_{sim}$. For each $\mathcal{F}_t$, we train multiple sets of $\mathcal{F}_s$ and $\mathcal{F}_i$ using different architectures. $\mathcal{V}_er$ queries $\mathcal{F}_t$, $\mathcal{F}_s$ and $\mathcal{F}_i$ with $\mathcal{D}_v$ to generate embeddings. Each node in $\mathcal{D}_v$ will thus have three corresponding embeddings from each GNN, denoted as h_t , h_s , and h_i , respectively. $\mathcal{V}_er$ generates a distance vector between h_t and h_s , representing a positive (similar) data point. In contrast, the distance vector between h_t and h_i is represented as a negative (not similar) data point. The distance vector is the element-wise squared distance between the two embeddings. This allows $\mathcal{V}_er$ to generate many data points from one pair of models equal to the size of $\mathcal{D}_v$. $\mathcal{V}_er$ then uses this data to optimize $\mathcal{C}_{sim}$ to classify whether a pair of embeddings is similar.

Verification. Once $\mathcal{C}_{sim}$ has been trained $\mathcal{V}_er$ can use it to make inferences about $\mathcal{F}_?$. $\mathcal{V}_er$ queries both $\mathcal{F}_?$ and $\mathcal{F}_t$ with $\mathcal{D}_v$ to generate embeddings. The distance vector is calculated between corresponding embeddings and passed to $\mathcal{C}_{sim}$, which outputs whether the pair is similar. If more than 50% of the pairs of embeddings are classified as similar, $\mathcal{V}_er$ classifies $\mathcal{F}_?$ as a surrogate, and independent otherwise. In our experiments we found that the precise threshold does not matter. In general, for $(\mathcal{F}_t, \mathcal{F}_s)$ pairs, $\approx 90\%$ of the embeddings are similar, while for $(\mathcal{F}_t, \mathcal{F}_i)$ pairs, only $\approx 10\%$ embeddings are classified as similar.

6. Experimental Setup

We evaluate GROVE using six datasets, three model architectures, and two model extraction attacks. We describe the datasets and their splits (Section 6.1), model architectures (Section 6.2), metrics for evaluation (Section 6.3), and the model extraction attacks (Section 6.4).

6.1. Datasets

Following prior work [27], we consider six benchmark graph datasets representing different types of graph networks. We describe the details of each of these datasets below.

DBLP [66] is a citation network where the 17,716 nodes represent publications and 105,734 edges indicate citations between different publications. Each node has 1,639 features based on the keywords in the paper. This is a node classification problem with four classes indicating the publication category.

CITeseer [67] is a citation network where the 4,120 nodes represent publications and 5,358 edges indicate citations between different publications. Each node has 602 features indicating the absence/presence of the corresponding word from the dictionary. This is a node classification task where the publications are categorized into six classes.

PUBMED [68] is a citation network where the 19,717 nodes represent publications and 88,648 edges indicate citations between different publications. Each node has 500 features described by a TF/IDF weighted word vector from a dictionary which consists of 500 unique words. This is a node classification task where the publications are categorized into three classes.

COAUTHOR [69] is a co-authorship network where the 34,493 nodes represent different authors, which are connected with 495,924 edges if they coauthored a paper. Here, the 8,415 node features represent paper keywords for each author’s papers. This node classification task is to predict the most active field of study out of five possibilities for each author.

ACM [70] is a heterogeneous graph that contains 3025 papers published in KDD, SIGMOD, SIGCOMM, Mobi-COMM, and VLDB. Papers that the same author publishes have an edge between them, resulting in 26,256 edges. Each paper is divided into three classes (Database, Wireless Communication, Data Mining), and the 1,870 features for each node are the bag-of-words representation of their keywords. **AMAZON** [71] is an abbreviation for Amazon Co-purchase Network for Photos. The 7,650 nodes represent items, and the 143,663 edges indicate whether the two items are bought together. Each of the 745 nodes features are bag-of-words encoded product reviews. Each of the items is classified into one of eight product categories.

Dataset Splits for Model Extraction. We split the dataset into two disjoint sets, each 40% of the dataset. One chunk is used to train $\mathcal{F}_t$, and the other chunk is used to train $\mathcal{F}_s$. While the original model extraction attacks use overlapping and non-overlapping training data for $\mathcal{F}_s$, we choose a non-overlapping dataset since that is the most challenging case for $\mathcal{V}_r$, as $\mathcal{F}_s$ is distinct from $\mathcal{F}_t$. For the same reason, $\mathcal{F}_t$ and $\mathcal{F}_i$ are trained on the same set. We evaluate $\mathcal{F}_t$ and $\mathcal{F}_s$ on a test set that is 10% of the dataset. Finally, $\mathcal{D}_v$ is the remaining 10%. The dataset sizes are shown in Table 2

TABLE 2. DATA SPLITS FOR TRAINING AND EVALUATING DIFFERENT MODELS.

Dataset	$\mathcal{F}_t$ Train	$\mathcal{F}_s$ Train	Test	$\mathcal{D}_v$
CoAUTHOR	13797	13797	3449	3449
PUBMED	7887	7887	1972	1972
DBLP	7086	7086	1772	1772
AMAZON	3060	3060	765	765
CITSEER	1648	1648	412	412
ACM	1210	1210	303	303

6.2. Model Architectures

Following prior work [27], we use the GAT, GIN, and GraphSAGE architectures (Section 2.1). For all architectures, the hidden layer size is 256, and the final hidden layer is connected to a dense layer for classification. The embeddings are extracted from the last hidden layer. We use the cross-entropy loss as the node classification loss, the ReLU activation function, and the Adam optimizer with an initial learning rate of 0.001. All models are trained for 200 epochs with early stopping based on validation accuracy.

GIN. We use a three-layer GIN model. The neighborhood sample size is fixed at ten samples at each layer.

GAT. We use a three-layer GAT model with a fixed neighborhood sample size of 10 at each layer. The first and second layers have four attention heads each.

GraphSAGE. We use a two-layer GraphSAGE model. The neighborhood sample sizes are set to 25 and 10, respectively. Following prior work [2], the MEAN aggregation function is used at each layer, and the dropout is set to 0.5 to prevent overfitting.

Similarity Model. We use the Scikit-Learn implementation of an MLPClassifier to train $\mathcal{C}_{sim}$. We train three versions of $\mathcal{F}_t$ for each dataset using the three architectures. Each

$\mathcal{F}_t$ has its own $\mathcal{C}_{sim}$. We train three versions of $\mathcal{F}_i$ and two versions of $\mathcal{F}_s$ for each $\mathcal{F}_t$ which we use to generate the positive and negative data points as explained in Section 5. We use a two-layer MLP and find the best hyperparameters using a grid search. The hidden layer sizes in the search are either 64 or 128, and the activation function is either Tanh or ReLU. We select the best model based on 10-fold cross-validation. We only train $\mathcal{C}_{sim}$ on Type I attacks unless otherwise stated.

To evaluate $\mathcal{C}_{sim}$, we train nine different versions of $\mathcal{F}_i$ and $\mathcal{F}_s$, i.e., 45 test models with different random initialization. These additional models are used to ensure the statistical significance of the results. We ran each experiment five times and reported the average with a 95% confidence interval.

6.3. Metrics

Following prior work [20], [27], we use two metrics for evaluating the effectiveness of model extraction attacks: accuracy and fidelity.

Accuracy measures the number of predictions $\mathcal{F}_s$ classifies correctly, compared to the ground-truth.

Fidelity measures the label agreement between $\mathcal{F}_t$ and $\mathcal{F}_s$.

To evaluate the effectiveness of GROVE, we use two additional metrics:

False Positive Rate (FPR) is the ratio of false positives to the sum of false positives and true negatives. This indicates the fraction of independent models incorrectly classified as surrogate.

False Negative Rate (FNR) is the ratio of false negatives to the sum of false negatives and true positives. This indicates the fraction of surrogate models incorrectly classified as independent.

6.4. Model Extraction Attack

We use Shen et al.’s [27] model extraction attack against inductive GNNs from their source code: <https://github.com/xinleihe/GNNStealing>. The details of the attack have been explained in Section 2.2. As mentioned in Section 6.1, we train $\mathcal{F}_s$ on $\mathcal{D}_s$ that is disjoint from $\mathcal{D}_t$. All the hyperparameters are set to the default values presented in the original paper. The performance of these models is summarized in Table 3, and our results are similar to those reported in the original attack paper.

7. Evaluation of GROVE

We show that GROVE satisfies the requirements outlined in Section 3.3. To this end, we evaluate GROVE’s effectiveness (requirement **R2**) in differentiating between $\mathcal{F}_s$, derived from the two types of model extraction attacks, and $\mathcal{F}_i$ (Section 7.1). We evaluate GROVE’s robustness (requirement **R3**) to $\mathcal{Adv}\mathcal{R}$ ’s attempts at evading verification (Section 7.2). Finally, we evaluate GROVE’s efficiency (requirement **R4**) (Section 7.3).

TABLE 3. AVERAGE ACCURACY AND FIDELITY VALUES (WITH 95% CONFIDENCE INTERVALS) OF $\mathcal{F}_t$, $\mathcal{F}_i$, AND $\mathcal{F}_s$ USED IN THE EVALUATION. VALUES ARE AVERAGED ACROSS MULTIPLE ARCHITECTURES (SECTION 6.2).

Dataset	$\mathcal{F}_t$ Accuracy	$\mathcal{F}_i$ Accuracy	Type I $\mathcal{F}_s$ Accuracy	Type I $\mathcal{F}_s$ Fidelity	Type II $\mathcal{F}_s$ Accuracy	Type II $\mathcal{F}_s$ Fidelity
ACM	0.906 $\pm$ 0.025	0.919 $\pm$ 0.021	0.888 $\pm$ 0.019	0.931 $\pm$ 0.019	0.896 $\pm$ 0.010	0.954 $\pm$ 0.020
AMAZON	0.879 $\pm$ 0.064	0.876 $\pm$ 0.050	0.861 $\pm$ 0.022	0.870 $\pm$ 0.051	0.842 $\pm$ 0.007	0.848 $\pm$ 0.009
CITSEER	0.804 $\pm$ 0.047	0.809 $\pm$ 0.028	0.757 $\pm$ 0.014	0.907 $\pm$ 0.041	0.796 $\pm$ 0.000	0.902 $\pm$ 0.012
CoAUTHOR	0.926 $\pm$ 0.005	0.928 $\pm$ 0.011	0.919 $\pm$ 0.019	0.949 $\pm$ 0.034	0.919 $\pm$ 0.004	0.948 $\pm$ 0.003
DBLP	0.696 $\pm$ 0.028	0.693 $\pm$ 0.030	0.674 $\pm$ 0.009	0.833 $\pm$ 0.018	0.680 $\pm$ 0.008	0.851 $\pm$ 0.017
PUBMED	0.846 $\pm$ 0.022	0.846 $\pm$ 0.021	0.829 $\pm$ 0.007	0.923 $\pm$ 0.016	0.832 $\pm$ 0.005	0.937 $\pm$ 0.014

7.1. Effectiveness

TABLE 4. PERFORMANCE OF GROVE AGAINST TYPE 1 AND TYPE 2 ATTACKS. AVERAGE FPR AND FNR VALUES ARE REPORTED ACROSS 5 EXPERIMENTS WITH 95% CONFIDENCE INTERVALS.

Dataset	FPR	Type 1 FNR	Type 2 FNR
ACM	0.022 $\pm$ 0.022	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000
AMAZON	0.034 $\pm$ 0.029	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000
CITSEER	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000
CoAUTHOR	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000
DBLP	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000
PUBMED	0.002 $\pm$ 0.002	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000

We first show that GROVE is effective at distinguishing between $\mathcal{F}_i$ and $\mathcal{F}_s$ (requirement **R2**). To this end, we train three $\mathcal{F}_i$ models and two $\mathcal{F}_s$ models using the Type 1 attack to train $\mathcal{C}_{sim}$. During the testing phase of $\mathcal{C}_{sim}$, we train additional $\mathcal{F}_i$ models to compute the FPR and $\mathcal{F}_s$ models from both Type 1 and Type 2 attacks to compute the FNR.

Ideally, we expect GROVE to have low FPR and FNR while differentiating between $\mathcal{F}_i$ and $\mathcal{F}_s$ using $\mathcal{C}_{sim}$. As seen in Table 4, we find that GROVE indeed has zero false negatives against both model extraction attacks. We observe close to zero false positives across four datasets (DBLP, PUBMED, CITSEER, and CoAUTHOR). For the AMAZON and ACM datasets, we note a low false positive rate of 3.4% and 2.2%, respectively.

These results show that GROVE is effective at distinguishing between $\mathcal{F}_s$ and $\mathcal{F}_i$ across different datasets and different architectures, satisfying **R2**.

7.2. Adversarial Robustness of GROVE

We now discuss the robustness of GROVE against attempts to evade detection by $Adv.\mathcal{R}$ (requirement **R3**). $Adv.\mathcal{R}$ can evade detection by differentiating $\mathcal{F}_s$ from $\mathcal{F}_t$ via (1) simple evasion or (2) model retraining.

Simple evasion techniques can be used by $Adv.\mathcal{R}$ to evade detection without retraining $\mathcal{F}_s$: (a) *model replacement* and (b) *post-processing*. These are described below.

Model replacement occurs when during the verification process, Ver intends to query the model that *Responder* has deployed ($\mathcal{F}_s$), but $Adv.\mathcal{R}$ replaces it by an independent model $\mathcal{F}_i$ to deceive Ver . Our system model (Section 3.1) pre-empts this by requiring *Responder* to register $\mathcal{F}_s$ before deployment. During verification, Ver conducts a fidelity check between the outputs of the registered and deployed

models (i.e., both outputs should match perfectly) to confirm that they are the same. As both models should be identical, the fidelity score between embeddings of the same input should be perfect.

Post-processing occurs when $Adv.\mathcal{R}$ applies a (linear) transformation on $\mathcal{H}_s$ to change its distribution while maintaining utility. If the distances between the embeddings of any two nodes of an input graph remain relatively the same after the transformation, it will not affect the result of any downstream task. Our system model pre-empts this by requiring $Adv.\mathcal{R}$ to send $\mathcal{F}_s$ to Ver , who verifies whether the outputs are generated directly from $\mathcal{F}_s$ without being post-processed. Furthermore, Ver can inspect $\mathcal{F}_s$ to ensure there are no non-standard layers that arbitrarily transform the output (Step 3 of the Verification Process in Section 3.1).

Model retraining. We identify four possible techniques from prior work to evade detection via model retraining [31], [36]: (a) *fine-tuning*, (b) *double extraction*, (c) *pruning* and (d) *distribution-shift*.

Fine-tuning retrains a previously trained model on a new dataset to improve model performance or change the classification task by replacing the model’s final layer. While this is popular in prior work [31], [32], [36], we argue that it cannot be used by $Adv.\mathcal{R}$ to evade detection of extracted GNN models. Recall from Section 6.4 that $\mathcal{F}_s$ consists of two independent components: the first outputs embeddings, and the second outputs class labels. Recalling (6), the embeddings are updated using an MSE loss with embeddings from $\mathcal{F}_t$. Traditional fine-tuning based on different class labels will only update the classifier, not affecting the embeddings.

Thus, we only test GROVE against end-to-end fine-tuning, where $Adv.\mathcal{R}$ updates both the described components while fine-tuning on a separate dataset from the same distribution. The average performance of the models is reported in Table 5. We found that $\mathcal{F}_s$ accuracy improves slightly after end-to-end fine-tuning while the fidelity slightly decreases. Despite this change, GROVE still achieves zero false negatives across all datasets, showing GROVE is effective in mitigating end-to-end fine-tuning attacks across different datasets.

Double extraction involves $Adv.\mathcal{R}$ running two model extraction attacks to obtain the final $\mathcal{F}_s$: first against $\mathcal{F}_t$ to get an intermediate model, followed by another attack against the intermediate model to obtain $\mathcal{F}_s$. This additional attack is to make the $\mathcal{F}_s$ distinct from $\mathcal{F}_t$. We refer to such surrogates as $\mathcal{F}_{s2}$

TABLE 5. PERFORMANCE OF END-TO-END FINE-TUNING. IN MANY CASES $\mathcal{F}_s$ ACCURACY IS HIGHER, BUT THE FIDELITY IS LOWER. FOR COAUTHOR THE ACCURACY OF $\mathcal{F}_s$ SURPASSED $\mathcal{F}_t$ AFTER FINE-TUNING.

Dataset	Type 1 $\mathcal{F}_s$ Accuracy	Type 1 $\mathcal{F}_s$ Fidelity	Type 2 $\mathcal{F}_s$ Accuracy	Type 2 $\mathcal{F}_s$ Fidelity
ACM	0.899 ± 0.030	0.927 ± 0.041	0.902 ± 0.013	0.939 ± 0.021
AMAZON	0.875 ± 0.018	0.874 ± 0.029	0.866 ± 0.021	0.863 ± 0.032
CITeseer	0.787 ± 0.018	0.838 ± 0.018	0.765 ± 0.017	0.834 ± 0.031
CoAUTHOR	0.935 ± 0.005	0.939 ± 0.005	0.937 ± 0.006	0.940 ± 0.003
DBLP	0.706 ± 0.010	0.706 ± 0.021	0.708 ± 0.011	0.724 ± 0.033
PUBMED	0.832 ± 0.010	0.924 ± 0.007	0.841 ± 0.007	0.935 ± 0.011

To satisfy the robustness experiment, GROVE should achieve low FNR against double extraction even if $\mathcal{F}_{s2}$ accuracy drops by up to 5% points. An accuracy drop greater than 5% points greatly reduces model utility. We use the previously trained $\mathcal{C}_{sim}$ directly and build additional $\mathcal{F}_{s2}$ models on the previously trained $\mathcal{F}_t$ models. We use the same attack for both extractions since the difference between the two attacks is the knowledge of $\mathcal{Adv.R}$, and it is unrealistic for $\mathcal{Adv.R}$ to have differing knowledge when running two attacks sequentially. As before, we use two architectures for each $\mathcal{F}_{s2}$ per $\mathcal{F}_t$, and create nine versions using random initialization (a total of 18 test models per $\mathcal{F}_t$). We repeat each experiment five times.

We evaluated GROVE against $\mathcal{F}_{s2}$ models reported in Table 6. We find that while the adversary experiences a significant loss in utility (at least 3% points in accuracy) on ACM, AMAZON, and CITeseer datasets, GROVE still achieves zero false negatives across all datasets. This shows that GROVE is effective in mitigating double extraction attacks across different datasets.

Pruning removes model weights to reduce computational complexity while maintaining model utility. This alters the output distribution (in our case, embeddings), which could potentially affect the success of GROVE.

We experiment with *prune ratios* (ratio of all model weights set to 0) ranging from 0.1 to 0.7 as pruning beyond resulted in a high accuracy loss (>20% points for all datasets). As before, GROVE is robust if it achieves low FNR against $\mathcal{F}_s$ models with less than a 5% point drop in accuracy. We use the same experimental setup of training nine versions of $\mathcal{F}_s$ and pruning each one with the ratios above (a total of 18 test models per prune ratio).

We report our results in Figure 4. We observe that $\mathcal{F}_s$ accuracy falls significantly after 0.4 (>5% points for all datasets). Only the FNR for CoAUTHOR stays small until a ratio of 0.6. We conjecture this is due to CoAUTHOR being the largest dataset, it has 1.8 times more nodes and 5 times more edges than the second largest dataset. For the rest of the datasets the FNR increases around a ratio of 0.2 to 0.3. This shows that GROVE in its basic form fails against pruning.

GROVE can be made robust against pruning attacks by adversarially optimizing $\mathcal{C}_{sim}$ (Section 5). We do this by including the output from pruned models up to a ratio of 0.4 into the training data. Beyond a prune ratio of 0.4, the accuracy drop is large enough (> 5% points) to deter $\mathcal{Adv.R}$. Using the same experimental setup, we evaluate the success of the more robust GROVE to mitigate pruning.

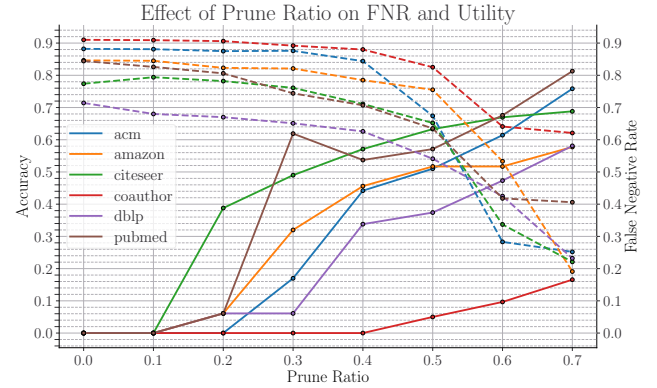


Figure 4. GROVE performance against pruning. Dotted lines represent $\mathcal{F}_s$ accuracy, and solid lines represent FNR. As the pruning ratio increases, the accuracy decreases, and the FNR increases. By default, GROVE fails against prune ratios of 0.3-0.4 for most datasets.

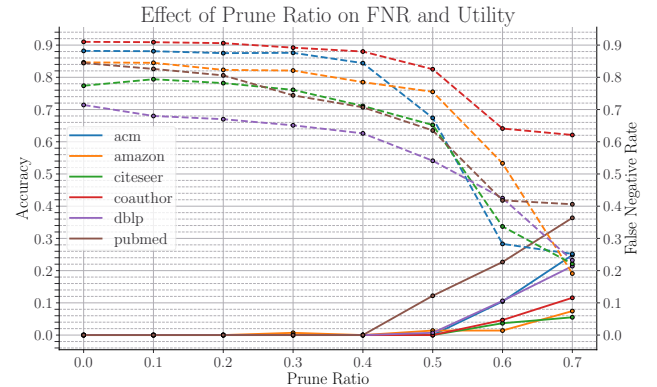


Figure 5. A more robust GROVE performs better against pruning. Dotted lines represent $\mathcal{F}_s$ accuracy, and solid lines represent FNR. Robust-GROVE works up till a prune ratio of 0.4, after which $\mathcal{F}_s$ utility decreases more than 5%.

We observe that GROVE still achieves zero false negatives against the basic and double extraction attacks mentioned before. Additionally, it achieves a lower FPR, with only a 1.4% FPR for AMAZON, 0.7% FPR for PUBMED, and a 0% FPR for the other datasets. We report the results for pruning in Figure 5. It achieves nearly zero FNR for all datasets up to a prune ratio of 0.4. Noting the success of robust optimization, we conjecture that this can be used to make GROVE robust against future evasion techniques.

TABLE 6. PERFORMANCE OF $\mathcal{F}_{s2}$. THE MODEL UTILITY IS COMPARABLE TO $\mathcal{F}_s$ IN MOST CASES, BUT DROPS BY $\approx 8\%$ POINTS FOR AMAZON AND $\approx 12\%$ POINTS FOR CITESEER.

Dataset	Type 1 $\mathcal{F}_{s2}$ Accuracy	Type 1 $\mathcal{F}_{s2}$ Fidelity	Type 2 $\mathcal{F}_{s2}$ Accuracy	Type 2 $\mathcal{F}_{s2}$ Fidelity
ACM	0.876 ± 0.023	0.932 ± 0.018	0.882 ± 0.017	0.930 ± 0.020
AMAZON	0.780 ± 0.046	0.782 ± 0.056	0.698 ± 0.216	0.695 ± 0.219
CITESEER	0.681 ± 0.086	0.719 ± 0.086	0.679 ± 0.064	0.736 ± 0.093
COAUTHOR	0.918 ± 0.006	0.943 ± 0.011	0.916 ± 0.009	0.943 ± 0.004
DBLP	0.686 ± 0.009	0.786 ± 0.027	0.678 ± 0.019	0.784 ± 0.036
PUBMED	0.829 ± 0.005	0.923 ± 0.007	0.831 ± 0.004	0.930 ± 0.005

We, therefore, conclude that GROVE remains effective (R2) even in the presence of adversaries (R3). Furthermore, adversarially training GROVE allows $\mathcal{V}_e$ to choose between resource utilization and performance. The larger and more diverse the training data for $\mathcal{C}_{sim}$, the longer it would take to train GROVE, and the more robust it would be. We have shown the minimum training data required to perform well against the evasion techniques we identified.

Distribution-shift is a method that allows $\mathcal{A}_v$ to adversarially train $\mathcal{F}_s$ to change the output distribution to make the embeddings distinct from $\mathcal{F}_t$'s embeddings. This is constrained to minimize degradation of $\mathcal{F}_s$ accuracy.

We designed an experiment to reproduce this by training $\mathcal{F}_s$ with an adversarial autoencoder that shifts the output distribution of $\mathcal{F}_s$ to a Gaussian distribution. In this, $\mathcal{F}_s$ is treated as a generator and is trained simultaneously with a discriminator that detects the Gaussian distribution. We could not successfully modify the output distribution of $\mathcal{F}_s$, and GROVE could successfully classify all such models with zero false negatives. We conjecture that the high-fidelity requirement forces the output distribution of $\mathcal{F}_s$ to be similar to $\mathcal{F}_t$.

7.3. Efficiency of GROVE

We now evaluate the efficiency of GROVE (requirement R4). We want to ensure the computation overhead of GROVE is reasonable.

To this end, we measure the execution time to generate the training data for $\mathcal{C}_{sim}$ and train it. Recall from Section 6.2 that we train multiple versions of $\mathcal{F}_i$ and $\mathcal{F}_s$ to generate the training data for $\mathcal{C}_{sim}$. For GROVE, this involves building six versions of $\mathcal{F}_i$ (two models per architecture with different random initializations) and two versions of $\mathcal{F}_s$ along with their pruned variants (ten versions of $\mathcal{F}_s$). These models can be trained in parallel, making it faster. However, we train them sequentially to accurately measure the total time taken, assuming just one available machine. The models were trained on a machine with two AMD EPYC 7302 16-Core CPUs and eight Nvidia A100 GPUs with 40GB VRAM per GPU.

We summarize the results for each target model in Table 7. It generally takes less than 3 hours to train GROVE. The time taken to train GROVE includes the time taken to train the additional surrogate and independent models to generate the training data and to train $\mathcal{C}_{sim}$. The size of the dataset influences the time taken to train the additional models. COAUTHOR is the largest dataset, with the

longest training times, followed by DBLP and PUBMED. The variation in time taken within datasets is caused by the optimization of $\mathcal{C}_{sim}$. For instance, for AMAZON, the time taken to train $\mathcal{C}_{sim}$ for GAT-based $\mathcal{F}_t$ was 187 seconds on average, while for GIN-based $\mathcal{F}_t$ it took 340 seconds on average. There is no trend for which architecture takes the longest time; it is affected by the combination of dataset and architecture. GROVE built for ACM-based models was the fastest to train, with $\mathcal{C}_{sim}$ training taking less than 100 seconds. For COAUTHOR, however, $\mathcal{C}_{sim}$ training took much longer, between 1561 to 2103 seconds.

Recall that in the practical setting we describe in Section 3.1, $\mathcal{V}_e$ only trains GROVE when $\mathcal{A}_v$ initiates a dispute with $\mathcal{R}$. Considering most models will not encounter ownership disputes, we consider the numbers in Table 7 to be reasonable.

In conclusion, we show that GROVE satisfies the requirement of being an effective (R2) robust (R3) and efficient (R4) fingerprinting scheme.

8. Related Work

In addition to the prior work directly related to GROVE (Section 2.2), we broadly discuss other related work pertaining to privacy and security in GNNs.

Robustness Attacks against GNNs find adversarial examples modifying the adjacency matrix to perturb the input, which results in misclassification [62], [63], [72]–[74]. As mentioned in Section 3.4, finding adversarial examples for graphs is not trivial. The best attack rate in these prior works is 35%. For comparison, image-based robustness attacks can reach nearly a 100% attack success rate [75].

Defences against Robustness Attacks limit the effect of perturbed edges on the GNN [72], [73], [76]–[79] or adversarial training (training with adversarial examples) to make the model robust [80]–[82]. Some defenses prune edges in the adjacency matrix to minimize the effect of the adversarial perturbations on edges with the intuition that homophily ensures nodes and edges connecting them are similar [72], [76]. Hence, any non-similar edges are potentially adversarial and can be pruned. This can also be extended to cases where homophily is not satisfied [77]. RobustGCN [83] trains the model such that the embeddings follow a Gaussian distribution to reduce the effect of perturbed edges.

Privacy Attacks against GNNs aim to infer sensitive unobservable information pertaining to training dataset and inputs given access to the model. Membership inference

TABLE 7. AVERAGE TOTAL TIME OVER FIVE RUNS IN SECONDS (WITH 95% CONFIDENCE INTERVALS) TAKEN TO TRAIN $\mathcal{F}_t$, AND GENERATE TRAINING DATA AND TRAIN $\mathcal{C}_{sim}$.

Dataset	GAT		Architectures		GraphSAGE	
	$\mathcal{F}_t$	$\mathcal{C}_{sim}$	$\mathcal{F}_t$	$\mathcal{C}_{sim}$	$\mathcal{F}_t$	$\mathcal{C}_{sim}$
COAUTHOR	1184 ± 53	10562 ± 1548	1060 ± 55	10668 ± 1205	855 ± 34	10550 ± 1237
PUBMED	435 ± 25	3961 ± 492	418 ± 26	3845 ± 257	374 ± 25	3856 ± 288
DBLP	459 ± 30	4182 ± 462	412 ± 26	4011 ± 498	397 ± 25	3730 ± 202
AMAZON	379 ± 26	3312 ± 218	361 ± 25	3273 ± 171	348 ± 21	3473 ± 323
CITSEER	389 ± 19	3312 ± 124	357 ± 24	3142 ± 204	349 ± 29	2970 ± 186
ACM	334 ± 27	2985 ± 165	343 ± 27	2943 ± 223	351 ± 33	2876 ± 134

attacks infer whether a node/sub-graph was used to train the model [84]–[87]. Olatunji et al. [84] further present a node-level membership inference attack using the 2-hop subgraph of the target node. He et al. [85] achieve a similar goal, except they use only the 0-hop subgraph. Wu et al. [87] present the first work on graph-level membership inference attack. Access to the embeddings can also allow adversaries to reconstruct the training graph [86]. Link inference attacks identify whether there exists an edge between two query nodes [88], [89]. Property inference attacks infer global properties about the training graph datasets (e.g., average clustering coefficient, the proportion of males and females) [90]–[92].

9. Discussion and Conclusions

We summarize our contributions and discuss some implications of our system model assumptions and design choices as well as potential limitations of GROVE.

9.1. Summary

In this paper, we proposed GROVE, a technique for fingerprinting GNNs and showed that it is effective against the state-of-the-art model extraction attack against GNNs described by Shen et al. [27]. Further, we showed that GROVE is robust against various ways in which an adversary can try to alter the fingerprint of the surrogate model.

9.2. Low-Fidelity Model Extraction Attacks

Recall that Shen et al.’s [27] attack aims to maximize fidelity between $\mathcal{F}_t$ and $\mathcal{F}_s$. Consequently, GROVE relies on the fact that the embeddings of $\mathcal{F}_t$ and $\mathcal{F}_s$ are similar. Low-fidelity extraction attacks will not necessarily result in $\mathcal{F}_s$ being close to $\mathcal{F}_t$, and can be potentially missed by GROVE. However, there are no low-fidelity GNN extraction attacks in the current literature.

9.3. Requirement for Model Registration

Following prior work [44], [51], [93], in our system model (Section 3.1), all model owners are required to obtain secure certified timestamps for their models before deployment to protect against different types of adversarial behavior. Model owners can be incentivized to do so because:

- registration serves as a direct means of protecting models against theft,
- it may be necessary to meet regulatory requirements for deployed models (e.g., EU guidelines [94]),
- in the future, model insurance mechanisms may be used to mitigate the effects of model theft, and model registration may become a requirement for insurance.

9.4. Revisiting *Adv.A*

In Section 3.1, we argued that *Adv.A* is thwarted by the model registration requirement. An alternative approach to trigger false positives in model ownership schemes is to generate the fingerprints adversarially. In all prior model ownership schemes, (both fingerprinting and watermarking) [31], [36], this is potentially feasible since it is the *Accuser* who generates the fingerprint/watermark. This is not the case in GROVE: the fingerprint is chosen by *Ver*, rendering GROVE immune to any such adversarial fingerprint generation attacks.

9.5. Shen et al.’s [27] Prediction-based Attack

In our adversary model (Section 3.2), we consider that $\mathcal{F}_t$ outputs embeddings as done in Shen et al. [27]. This allows us to use GROVE over the outputs of $\mathcal{F}_t$, making it a black-box scheme. However, Shen et al.’s [27] also suggest a prediction-based model extraction attack for cases where $\mathcal{F}_t$ only outputs classification labels. In such a setting, GROVE is still applicable if *Ver* is given white-box access to $\mathcal{F}_t$.

Acknowledgments

The authors would like to thank the anonymous reviewers for their valuable feedback. This work was supported in part by Intel (in the context of the Private-AI Institute).

References

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [2] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 1025–1035.

- [3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=ryGs6iA5Kmn>
- [4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=rJXMpikCZ>
- [5] L. Cai, J. Li, J. Wang, and S. Ji, "Line graph neural networks for link prediction," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5103–5113, 2021.
- [6] S. Yun, S. Kim, J. Lee, J. Kang, and H. J. Kim, "Neo-gnns: Neighborhood overlap-aware graph neural networks for link prediction," *Advances in Neural Information Processing Systems*, vol. 34, pp. 13 683–13 694, 2021.
- [7] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," *Advances in neural information processing systems*, vol. 31, 2018.
- [8] M. Zhang, P. Li, Y. Xia, K. Wang, and L. Jin, "Revisiting graph neural networks for link prediction," 2021. [Online]. Available: https://openreview.net/forum?id=8q_ca26L1fz
- [9] X. Wang, K. Yen, Y. Hu, and H.-W. Shen, "Deepgd: A deep learning framework for graph drawing using gnn," *IEEE computer graphics and applications*, vol. 41, no. 5, pp. 32–44, 2021.
- [10] C. Li, K. Jia, D. Shen, C. R. Shi, and H. Yang, "Hierarchical representation learning for bipartite graphs," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, 7 2019, pp. 2873–2879. [Online]. Available: <https://doi.org/10.24963/ijcai.2019/398>
- [11] J. Sun, Y. Zhang, W. Guo, H. Guo, R. Tang, X. He, C. Ma, and M. Coates, "Neighbor interaction aware graph convolution networks for recommendation," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1289–1298. [Online]. Available: <https://doi.org/10.1145/3397271.3401123>
- [12] X. Wang, X. He, M. Wang, F. Feng, and T.-S. Chua, "Neural graph collaborative filtering," in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 165–174. [Online]. Available: <https://doi.org/10.1145/3331184.3331267>
- [13] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 974–983. [Online]. Available: <https://doi.org/10.1145/3219819.3219890>
- [14] "Amazon Neptune ML - Easy, fast, and accurate predictions for graphs — aws.amazon.com," <https://aws.amazon.com/neptune/machine-learning>, [Accessed 26-07-2023].
- [15] A. Lerer, L. Wu, J. Shen, T. Lacroix, L. Wehrstedt, A. Bose, and A. Peysakhovich, "PyTorch-BigGraph: A Large-scale Graph Embedding System," in *Proceedings of the 2nd SysML Conference*, Palo Alto, CA, USA, 2019.
- [16] E. Rossi, F. Frasca, B. Chamberlain, D. Eynard, M. M. Bronstein, and F. Monti, "SIGN: scalable inception graph neural networks," *CoRR*, vol. abs/2004.11198, 2020. [Online]. Available: <https://arxiv.org/abs/2004.11198>
- [17] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction apis," in *Proceedings of the 25th USENIX Conference on Security Symposium*, ser. SEC'16. USA: USENIX Association, 2016, p. 601–618.
- [18] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, ser. ASIA CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 506–519. [Online]. Available: <https://doi.org/10.1145/3052973.3053009>
- [19] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff nets: Stealing functionality of black-box models," *CoRR*, vol. abs/1812.02766, 2018. [Online]. Available: <http://arxiv.org/abs/1812.02766>
- [20] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, *High Accuracy and High Fidelity Extraction of Neural Networks*. USA: USENIX Association, 2020.
- [21] N. Carlini, M. Jagielski, and I. Mironov, "Cryptanalytic extraction of neural network models," in *Advances in Cryptology – CRYPTO 2020*, D. Micciancio and T. Ristenpart, Eds. Cham: Springer International Publishing, 2020, pp. 189–218.
- [22] B. G. Atli, S. Szyller, M. Juuti, S. Marchal, and N. Asokan, "Extraction of complex dnn models: Real threat or boogeyman?" in *Engineering Dependable and Secure Machine Learning Systems*, O. Shehory, E. Farchi, and G. Barash, Eds. Cham: Springer International Publishing, 2020, pp. 42–57.
- [23] S. Szyller, V. Duddu, T. Gröndahl, and N. Asokan, "Good artists copy, great artists steal: Model extraction attacks against image translation generative adversarial networks," *CoRR*, vol. abs/2104.12623, 2021. [Online]. Available: <https://arxiv.org/abs/2104.12623>
- [24] S. Pal, Y. Gupta, A. Shukla, A. Kanade, S. Shevade, and V. Ganapathy, "A framework for the extraction of deep neural networks by leveraging public data," 2019. [Online]. Available: <https://arxiv.org/abs/1905.09165>
- [25] D. DeFazio and A. Ramesh, "Adversarial model extraction on graph neural networks," *arXiv preprint arXiv:1912.07721*, 2019.
- [26] B. Wu, X. Yang, S. Pan, and X. Yuan, "Model extraction attacks on graph neural networks: Taxonomy and realisation," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ser. ASIA CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 337–350. [Online]. Available: <https://doi.org/10.1145/3488932.3497753>
- [27] Y. Shen, X. He, Y. Han, and Y. Zhang, "Model stealing attacks against inductive graph neural networks," in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 1175–1192.
- [28] T. Lee, B. Edwards, I. Molloy, and D. Su, "Defending against neural network model stealing attacks using deceptive perturbations," in *2019 IEEE Security and Privacy Workshops (SPW)*, 2019, pp. 43–49.
- [29] M. Juuti, S. Szyller, A. Dmitrenko, S. Marchal, and N. Asokan, "PRADA: protecting against DNN model stealing attacks," *CoRR*, vol. abs/1805.02628, 2018. [Online]. Available: <http://arxiv.org/abs/1805.02628>
- [30] M. Kesarwani, B. Mukhoty, V. Arya, and S. Mehta, "Model extraction warning in mlaas paradigm," in *Proceedings of the 34th Annual Computer Security Applications Conference*, ser. ACSAC '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 371–380. [Online]. Available: <https://doi.org/10.1145/3274694.3274740>
- [31] N. Lukas, Y. Zhang, and F. Kerschbaum, "Deep neural network fingerprinting by conferrable adversarial examples," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021. [Online]. Available: <https://openreview.net/forum?id=VqzVhqxjH1>
- [32] P. Maini, M. Yaghini, and N. Papernot, "Dataset inference: Ownership resolution in machine learning," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=hvdKKV2yt7T>
- [33] J. Zhao, Q. Hu, G. Liu, X. Ma, F. Chen, and M. M. Hassan, "Afa: Adversarial fingerprinting authentication for deep neural networks," *Computer Communications*, vol. 150, pp. 488–497, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S014036641931686X>

- [34] X. Cao, J. Jia, and N. Z. Gong, "Ipguard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary," in *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 14–25. [Online]. Available: <https://doi.org/10.1145/3433210.3437526>
- [35] S. Wang and C.-H. Chang, "Fingerprinting deep neural networks - a deepfool approach," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.
- [36] Z. Peng, S. Li, G. Chen, C. Zhang, H. Zhu, and M. Xue, "Fingerprinting deep neural networks globally via universal adversarial perturbations," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 2022, pp. 13 420–13 429. [Online]. Available: <https://doi.org/10.1109/CVPR52688.2022.01307>
- [37] Y. Zheng, S. Wang, and C.-H. Chang, "A dnn fingerprint for non-repudiable model ownership identification and piracy detection," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2977–2989, 2022.
- [38] K. Krishna, G. S. Tomar, A. P. Parikh, N. Papernot, and M. Iyyer, "Thieves on sesame street! model extraction of bert-based apis," in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=Byl5NREFDr>
- [39] H. Hu and J. Pang, "Stealing machine learning models: Attacks and countermeasures for generative adversarial networks," in *Annual Computer Security Applications Conference*, ser. ACSAC '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1–16. [Online]. Available: <https://doi.org/10.1145/3485832.3485838>
- [40] E. Wallace, M. Stern, and D. Song, "Imitation attacks and defenses for black-box machine translation systems," *arXiv preprint arXiv:2004.15015*, 2020.
- [41] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy (SP)*, 2017, pp. 3–18.
- [42] Y. Uchida, Y. Nagai, S. Sakazawa, and S. Satoh, "Embedding watermarks into deep neural networks," *CoRR*, vol. abs/1701.04082, 2017. [Online]. Available: <http://arxiv.org/abs/1701.04082>
- [43] T. Wang and F. Kerschbaum, "RIGA: covert and robust white-box watermarking of deep neural networks," in *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*, J. Leskovec, M. Grobelnik, M. Najork, J. Tang, and L. Zia, Eds. ACM / IW3C2, 2021, pp. 993–1004. [Online]. Available: <https://doi.org/10.1145/3442381.3450000>
- [44] Y. Adi, C. Baum, M. Cisse, B. Pinkas, and J. Keshet, "Turning your weakness into a strength: Watermarking deep neural networks by backdoor," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 1615–1631. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/adi>
- [45] E. Merrer, P. Perez, and G. Trédan, "Adversarial frontier stitching for remote neural network watermarking," *Neural Computing and Applications*, vol. 32, 07 2020.
- [46] J. Guo and M. Potkonjak, "Watermarking deep neural networks for embedded systems," in *Proceedings of the International Conference on Computer-Aided Design*, ser. ICCAD '18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3240765.3240862>
- [47] A. Sablayrolles, M. Douze, C. Schmid, and H. Jegou, "Radioactive data: tracing through training," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 8326–8335. [Online]. Available: <https://proceedings.mlr.press/v119/sablayrolles20a.html>
- [48] L. Zhu, Y. Li, X. Jia, Y. Jiang, S.-T. Xia, and X. Cao, "Defending against model stealing via verifying embedded external features," in *ICML 2021 Workshop on Adversarial Machine Learning*, 2021. [Online]. Available: <https://openreview.net/forum?id=g6zfnWUG8A1>
- [49] H. Chen, B. D. Rouhani, C. Fu, J. Zhao, and F. Koushanfar, "Deepmarks: A secure fingerprinting framework for digital rights management of deep learning models," in *Proceedings of the 2019 International Conference on Multimedia Retrieval*, ser. ICMR '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 105–113. [Online]. Available: <https://doi.org/10.1145/3323873.3325042>
- [50] Z. He, T. Zhang, and R. Lee, "Sensitive-sample fingerprinting of deep neural networks," in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4724–4732.
- [51] S. Szyller, B. G. Atli, S. Marchal, and N. Asokan, "Dawn: Dynamic adversarial watermarking of neural networks," in *Proceedings of the 29th ACM International Conference on Multimedia*, ser. MM '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 4417–4425. [Online]. Available: <https://doi.org/10.1145/3474085.3475591>
- [52] N. Lukas, E. Jiang, X. Li, and F. Kerschbaum, "Sok: How robust is image classification deep neural network watermarking? (extended version)," *CoRR*, vol. abs/2108.04974, 2021. [Online]. Available: <https://arxiv.org/abs/2108.04974>
- [53] J. Xu and S. Picek, "Watermarking graph neural networks based on backdoor attacks," *arXiv preprint arXiv:2110.11024*, 2021.
- [54] X. Zhao, H. Wu, and X. Zhang, "Watermarking graph neural networks by random graphs," in *2021 9th International Symposium on Digital Forensics and Security (ISDFS)*. IEEE, 2021, pp. 1–6.
- [55] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 619–631. [Online]. Available: <https://doi.org/10.1145/3133956.3134056>
- [56] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "Gazelle: A low latency framework for secure neural network inference," in *Proceedings of the 27th USENIX Conference on Security Symposium*, ser. SEC'18. USA: USENIX Association, 2018, p. 1651–1668.
- [57] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun, "Scaling up trustless DNN inference with zero-knowledge proofs," *CoRR*, vol. abs/2210.08674, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.08674>
- [58] D. Boneh, W. Nguyen, and A. Ozdemir, "Efficient functional commitments: How to commit to a private function," *Cryptology ePrint Archive*, Paper 2021/1342, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1342>
- [59] F. Tramèr and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," in *International Conference on Learning Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=rJVorjCcKQ>
- [60] Y. Deng, C. Wang, S. Yu, S. Liu, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang, "Strongbox: A gpu tee on arm endpoints," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 769–783. [Online]. Available: <https://doi.org/10.1145/3548606.3560627>
- [61] S. Szyller, R. Zhang, J. Liu, and N. Asokan, "On the robustness of dataset inference," 2022. [Online]. Available: <https://arxiv.org/abs/2210.13631>
- [62] D. Zügner, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on neural networks for graph data," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 2847–2856.
- [63] H. Dai, H. Li, T. Tian, X. Huang, L. Wang, J. Zhu, and L. Song, "Adversarial attack on graph structured data," *CoRR*, vol. abs/1806.02371, 2018. [Online]. Available: <http://arxiv.org/abs/1806.02371>

- [64] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008. [Online]. Available: <http://jmlr.org/papers/v9/vandermaten08a.html>
- [65] G. Koch, R. Zemel, R. Salakhutdinov *et al.*, "Siamese neural networks for one-shot image recognition," in *ICML deep learning workshop*, vol. 2, no. 1. Lille, 2015.
- [66] S. Pan, J. Wu, X. Zhu, C. Zhang, and Y. Wang, "Tri-party deep network representation," in *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence*, ser. IJCAI'16. AAAI Press, 2016, p. 1895–1901.
- [67] C. Giles, K. Bollacker, and S. Lawrence, "Citeseer: an automatic citation indexing system," in *Proceedings of the ACM International Conference on Digital Libraries*. ACM, 1998, pp. 89–98, proceedings of the 1998 3rd ACM Conference on Digital Libraries ; Conference date: 23-06-1998 Through 26-06-1998.
- [68] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Magazine*, vol. 29, no. 3, p. 93, Sep. 2008. [Online]. Available: <https://ojs.aaai.org/index.php/aimagazine/article/view/2157>
- [69] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," *CoRR*, vol. abs/1811.05868, 2018. [Online]. Available: <http://arxiv.org/abs/1811.05868>
- [70] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, "Heterogeneous graph attention network," in *The World Wide Web Conference*, ser. WWW '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 2022–2032. [Online]. Available: <https://doi.org/10.1145/3308558.3313562>
- [71] J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, "Image-based recommendations on styles and substitutes," in *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 43–52. [Online]. Available: <https://doi.org/10.1145/2766462.2767755>
- [72] H. Wu, C. Wang, Y. Tyshetskiy, A. Docherty, K. Lu, and L. Zhu, "Adversarial examples for graph data: Deep insights into attack and defense," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*. International Joint Conferences on Artificial Intelligence Organization, 7 2019, pp. 4816–4823. [Online]. Available: <https://doi.org/10.24963/ijcai.2019/669>
- [73] N. Entezari, S. A. Al-Sayouri, A. Darvishzadeh, and E. E. Papalexakis, "All you need is low (rank) defending against adversarial attacks on graphs," in *Proceedings of the 13th International Conference on Web Search and Data Mining*, 2020, pp. 169–177.
- [74] D. Zügner, O. Borchert, A. Akbarnejad, and S. Günnemann, "Adversarial attacks on graph neural networks: Perturbations and their patterns," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 14, no. 5, pp. 1–31, 2020.
- [75] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, "Boosting adversarial attacks with momentum," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [76] X. Zhang and M. Zitnik, "Gnnguard: Defending graph neural networks against adversarial attacks," *Advances in neural information processing systems*, vol. 33, pp. 9263–9275, 2020.
- [77] C. Deng, X. Li, Z. Feng, and Z. Zhang, "GARNET: Reduced-rank topology learning for robust and scalable graph neural networks," in *The First Learning on Graphs Conference*, 2022. [Online]. Available: <https://openreview.net/forum?id=kvwWjYQtmw>
- [78] B. A. Miller, M. Çamurcu, A. J. Gomez, K. Chan, and T. Eliassi-Rad, "Improving robustness to attacks against vertex classification," in *MLG Workshop*, 2019.
- [79] Y. Zhang, S. Khan, and M. Coates, "Comparing and detecting adversarial attacks for graph deep learning," in *Proc. Representation Learning on Graphs and Manifolds Workshop, Int. Conf. Learning Representations, New Orleans, LA, USA*, 2019.
- [80] H. Jin and X. Zhang, "Latent adversarial training of graph convolution networks," in *ICML workshop on learning and reasoning with graph-structured representations*, vol. 2, 2019.
- [81] K. Sun, Z. Lin, H. Guo, and Z. Zhu, "Virtual adversarial training on graph convolutional networks in node classification," in *Pattern Recognition and Computer Vision: Second Chinese Conference, PRCV 2019, Xi'an, China, November 8–11, 2019, Proceedings, Part I 2*. Springer, 2019, pp. 431–443.
- [82] X. Tang, Y. Li, Y. Sun, H. Yao, P. Mitra, and S. Wang, "Transferring robustness for graph neural network against poisoning attacks," in *Proceedings of the 13th international conference on web search and data mining*, 2020, pp. 600–608.
- [83] D. Zhu, Z. Zhang, P. Cui, and W. Zhu, "Robust graph convolutional networks against adversarial attacks," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1399–1407. [Online]. Available: <https://doi.org/10.1145/3292500.3330851>
- [84] I. E. Olatunji, W. Nejdl, and M. Khosla, "Membership inference attack on graph neural networks," in *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 2021, pp. 11–20.
- [85] X. He, R. Wen, Y. Wu, M. Backes, Y. Shen, and Y. Zhang, "Node-level membership inference attacks against graph neural networks," 2023. [Online]. Available: <https://openreview.net/forum?id=XXKHnu4OF6wn>
- [86] V. Duddu, A. Boutet, and V. Shejwalkar, "Quantifying privacy leakage in graph embedding," in *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 2020, pp. 76–85.
- [87] B. Wu, X. Yang, S. Pan, and X. Yuan, "Adapting membership inference attacks to gnn for graph classification: Approaches and implications," in *2021 IEEE International Conference on Data Mining (ICDM)*, 2021, pp. 1421–1426.
- [88] X. He, J. Jia, M. Backes, N. Z. Gong, and Y. Zhang, "Stealing links from graph neural networks," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 2669–2686. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/he-xinlei>
- [89] Z. Zhang, M. Chen, M. Backes, Y. Shen, and Y. Zhang, "Inference attacks against graph neural networks," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 4543–4560. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/zhang-zhikun>
- [90] A. Suri and D. Evans, "Formalizing and estimating distribution inference risks," *CoRR*, vol. abs/2109.06024, 2021. [Online]. Available: <https://arxiv.org/abs/2109.06024>
- [91] A. Suri, Y. Lu, Y. Chen, and D. Evans, "Dissecting distribution inference," in *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2023.
- [92] X. Wang and W. H. Wang, "Group property inference attacks against graph neural networks," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2871–2884. [Online]. Available: <https://doi.org/10.1145/3548606.3560662>
- [93] J. Liu, R. Zhang, S. Szyller, K. Ren, and N. Asokan, "False claims against model ownership resolution," *arXiv preprint arXiv:2304.06607*, 2023.
- [94] "Home — artificialintelligenceact.eu," <https://artificialintelligenceact.eu/>, [Accessed 26-07-2023].

Appendix A. Dataset Ownership Graphs

The difference in the two models is most easy to see using COAUTHOR since that is the largest dataset. However, the same trend is seen in the rest of the plots, as shown in Figure 6. Due to space constraints, we only show the graphs for models trained with GAT. The same trend is seen across all architectures.

Appendix B. Model Ownership Graphs

We show the graphs for the model ownership experiment in Figure 7. Due to space constraints, we only show the graphs for models trained with GAT. The same trend is seen across all architectures.

Appendix C. Distance Graphs

We calculate the Euclidean distance between pairs of embeddings from $(\mathcal{F}_t, \mathcal{F}_s)$ and $(\mathcal{F}_t, \mathcal{F}_i)$ and plot the histogram of the distances in Figure 8. While the distribution of the distances is different, the distributions overlap significantly. Thus, a simple approach based on distance calculation and hypothesis testing did not work.

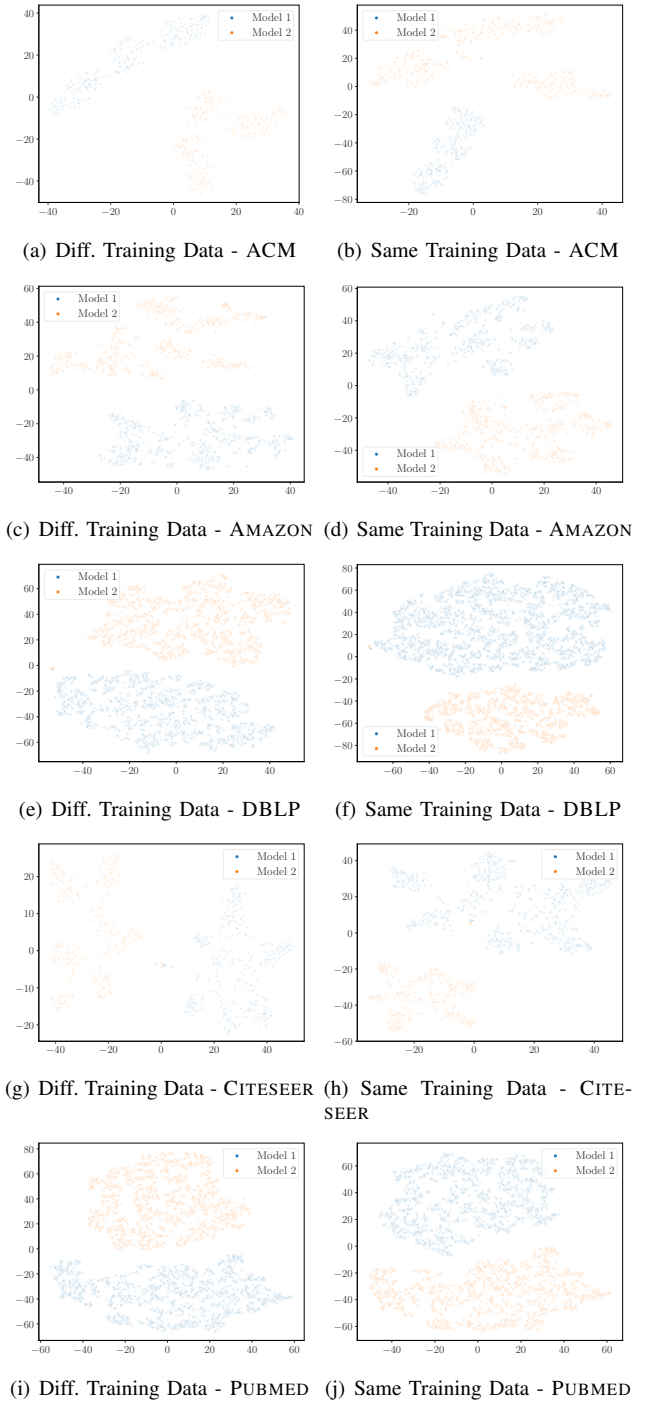
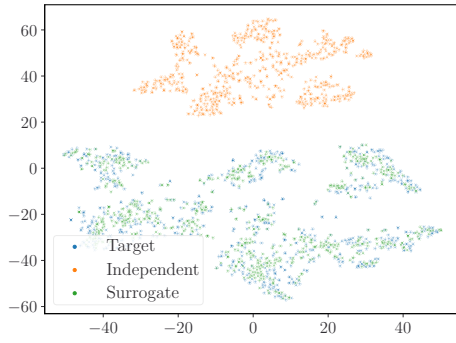


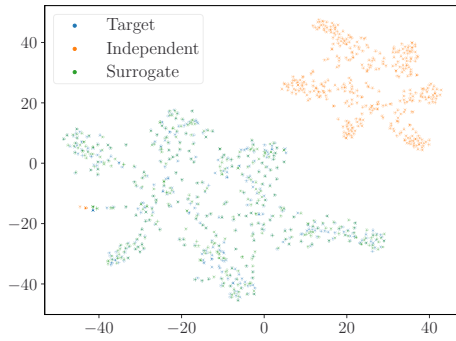
Figure 6. t-SNE projections of the embeddings from two models trained are distinguishable for all datasets, regardless of whether the architecture is the same or different.



(a) Models trained on ACM dataset



(b) Models trained on AMAZON dataset

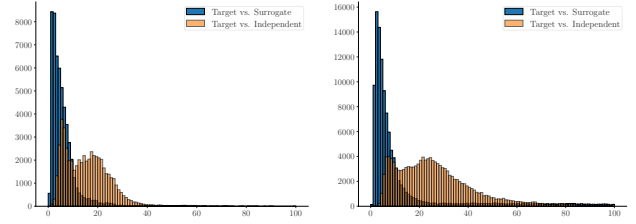


(c) Models trained on CITESEER dataset



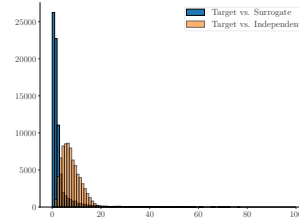
(d) Models trained on PUBMED dataset

Figure 7. t-SNE projections of the embeddings from $\mathcal{F}_s$ and $\mathcal{F}_t$ overlap, while those from $\mathcal{F}_i$ are distinct. All models in this figure use the GAT architecture. The same trend holds for all architectures.

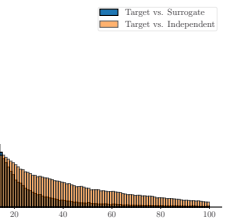


(a) ACM

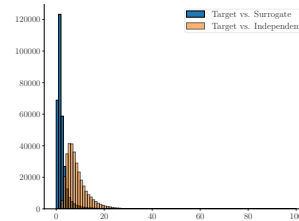
(b) AMAZON



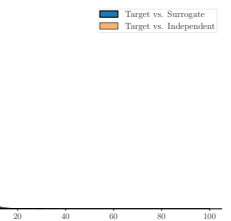
(c) CITESEER



(d) CoAUTHOR



(e) DBLP



(f) PUBMED

Figure 8. Histograms of the Euclidean distances between pairs of embeddings from $(\mathcal{F}_t, \mathcal{F}_s)$ and $(\mathcal{F}_t, \mathcal{F}_i)$.

Appendix D. Meta-Review

D.1. Summary

In this work the authors present the first fingerprinting mechanism for Graph Neural Networks (GNNs) to facilitate ownership verification.

D.2. Scientific Contributions

- Provides a Valuable Step Forward in an Established Field
- Addresses a Long-Known Issue
- Establishes a New Research Direction
- Creates a New Tool to Enable Future Science

D.3. Reasons for Acceptance

- 1) This paper provides a valuable step forward in an established field. The authors introduce the first fingerprinting mechanism to facilitate ownership verification in graph neural networks (GNNs), thus providing a novel defense against model extraction attacks in this domain.

- 2) This work addresses a long-known issue. State-of-the-art high-fidelity model stealing attacks are a real threat that has attracted significant research efforts. The mechanism introduced here can be used to defend against these attacks in GNN settings.
- 3) This work establishes a new research direction. As this is the first mechanism of its kind in GNNs, the authors have identified an area that requires further investigation and exploration. The unique approach introduced here (particularly the use of embeddings) can inspire future work in defending against model extraction and stealing attacks in GNNs.
- 4) This work provides a new tool capable of verifying ownership of a trained GNN model, which the authors plan to open source and make available for future research.

D.4. Noteworthy Concerns

- 1) The authors show that their approach is robust against an adversary's attempts at detection evasion. They do so by considering simple evasion attacks and model re-training. However, the robustness evaluation would be made considerably stronger by including (a) black box attacks and (b) adaptive attackers that could end-to-end fine tune the model to evade the proposed mechanism with, for example, an additional objective.
- 2) The authors note that it would not be feasible to empirically compare watermarking/fingerprinting solutions in other domains as it is with GrOVe in the graph domain. While the reviewers believe these baselines would still provide value, we also acknowledge that this is not a significant limitation as watermarking may affect training and other fingerprinting methods are not directly applied to GNNs.